

The first and third steps are things you already know how to do; you modify the fields in the model and migrate them. Django’s migration framework provides programmatic hooks that allow you to perform the second step as part of a migration. The `migrations` module has an operation class called `RunPython`. This class takes two references to functions: one function to be called when moving forward in a migration and another to be called when moving backward (yep, you can move backward—more on how to do that later).

Let’s de-Westernize `Promoter`. First, add the two new fields, and create the corresponding migration file. The following listing shows the state of `Promoter` after step 1.

#### Listing 10.6 Adding full and common name fields to the `Promoter` Model

```
# RiffMates/promoters/models.py
from django.db import models
```

```
class Promoter(models.Model):
    first_name = models.CharField(max_length=25)
    last_name = models.CharField(max_length=25)
    common_name = models.CharField(max_length=25)
    full_name = models.CharField(max_length=50)
    famous_for = models.CharField(max_length=50)
```

Add new common  
and full name  
fields to `Promoter`.

Note that the order you declare your fields in effects  
the underlying table structure but doesn’t have to  
correspond to when you added things to your model;  
`famous_for` can show up after the new fields.

With the change in place, create a new migration file by running the `makemigrations` management command. The resulting file has a rather unwieldy name: `0003_promoter_common_name_promoter_full_name.py`.

The second step in the de-Westernization process is the new one. Inside of a migration file, you use a `RunPython` class as an operation by providing code that copies data from the first and last name fields into the full and common name fields. This new code goes in a custom migration file. To create one, call `makemigrations` with the `--empty` flag. This creates a migration file stub with an automatically generated filename. My obsessive need for control over details means I can’t leave that alone, so I rename the file to be a little more clear. The process for creating and renaming the file is as follows:

```
(venv) RiffMates$ python manage.py makemigrations --empty promoters
Migrations for 'promoters':
  promoters/migrations/0004_auto_20230904_1552.py
(venv) RiffMates$ cd promoters/migrations/
(venv) migrations$ mv 0004_auto_20230904_1552.py 0004_data_fullname.py
(venv) RiffMates$ cd -
```

Create a stub migration file  
in the `promoters` app.

Rename the file something  
more descriptive.

The resulting stub migration file looks like this: